

Chapter 07

문자열 검색

브루트 포스 문자열 검색

텍스트의 각 위치에서 패턴을 처음부터 하나씩 대조하는 가장 기본적인 방법

txt = "ABABCDEFGHA"

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

pat = "ABC"

A	B	C
0	1	2

목표

텍스트 안에서 pat이 처음 시작되는 위치 찾기

핵심 원리

맞으면 계속 진행, 틀리면 시작 위치를 한 칸 오른쪽으로 옮김

정답: Python index 2 → 출력값 3

브루트 포스 코드의 전체 구조

브루트 포스법

```
def bf_match(txt, pat):  
    pt = 0  
    pp = 0  
  
    while pt != len(txt) and pp != len(pat):  
        if txt[pt] == pat[pp]:  
            pt += 1  
            pp += 1  
        else:  
            pt = pt - pp + 1  
            pp = 0  
    return pt - pp if pp == len(pat) else -1
```

```
s1 = "ABABCDEFGHA"  
s2 = "ABC"
```

pt
텍스트 txt에서 현재 비교 중인 위치

pp
패턴 pat에서 현재 비교 중인 위치

반복 조건
텍스트 끝 또는 패턴 끝에 도달할 때까지 비교함

실패 처리
비교 시작 위치를 한 칸 오른쪽으로 이동함

문자가 같으면 pt와 pp가 함께 이동함

txt

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

pat

A	B	C
0	1	2

1차 비교

txt[0] = A, pat[0] = A → 일치

pt: 0 → 1, pp: 0 → 1

2차 비교

txt[1] = B, pat[1] = B → 일치

pt: 1 → 2, pp: 1 → 2

```
if txt[pt] == pat[pp]:  
    pt += 1  
    pp += 1
```

현재까지 AB가 일치한 상태임

문자가 다르면 시작 위치를 다시 잡음

txt

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

pat

A	B	C
0	1	2

현재 비교
 $\text{txt}[2] = A$
 $\text{pat}[2] = C$
 $A \neq C \rightarrow \text{불일치}$

else:

$\text{pt} = \text{pt} - \text{pp} + 1$
 $\text{pp} = 0$

현재 $\text{pt} = 2, \text{pp} = 2$
 $\text{pt} = 2 - 2 + 1 = 1$
 $\text{pp} = 0$

기존 시작 위치 0 $\rightarrow$ 다음 시작 위치 1

왜 $pt = pt - pp + 1$ 인가?

$pt - pp$

방금 패턴 비교를 시작했던 텍스트 위치

$+ 1$

그 시작 위치에서 한 칸 오른쪽으로 이동

첫 번째 시도

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

$A = A, B = B, A \neq C$

실패 시 시작 위치 계산

pat

A	B	C
0	1	2

$pt = 2, pp = 2$

시작 위치 $= pt - pp = 0$

다음 시작 위치 $= 0 + 1 = 1$

브루트 포스는 재할용 없이 한 칸씩 다시 시작함

세 번째 시도에서 패턴 전체가 일치함

시도 1: 시작 위치 0

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

A	B	C
0	1	2

A, B는 맞지만 C에서 실패

시도 2: 시작 위치 1

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

A	B	C
0	1	2

txt[1]=B, pat[0]=A → 실패

시도 3: 시작 위치 2

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

A	B	C
0	1	2

txt[2:5] = ABC → pat 전체 일치

반환값과 핵심 정리

검색 성공 시
 $pp == \text{len}(\text{pat})$

최종 상태
 $pt = 5, pp = 3$

시작 위치
 $pt - pp = 5 - 3 = 2$

txt

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

pat

A	B	C
0	1	2

```
return pt - pp if pp == len(pat) else -1
```

```
idx = 2  
print(idx + 1) # 3
```

브루트 포스법: 틀리면 한 칸 옮겨 처음부터 다시 비교함

학습 목표

문자열 검색에서 "불필요한 재비교"가 왜 문제가 되는지 설명할 수 있음

KMP의 skip 배열이 의미하는 "앞부분과 뒷부분이 같은 최대 길이"를 설명할 수 있음

KMP 검색 과정에서 pt, pp가 어떻게 이동하는지 추적할 수 있음

보이어-무어법의 문자별 skip 표를 만들 수 있음

BM 검색 과정에서 오른쪽부터 비교하고, 이동량을 max()로 정하는 이유를 설명할 수 있음

두 알고리즘의 관점 차이를 비교할 수 있음

문자열 검색의 기본 문제

예시

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

txt = "ABABCDEF GHA"

pat = ABC

목표는 단순히 “있는지 없는지”가 아니라,
패턴이 텍스트의 몇 번째 위치에서 시작하는지를 찾는 것임

단순 검색은 실패할 때마다 거의 처음부터 다시 비교함
KMP와 BM은 실패 정보를 이용해 다음 비교 위치를 더 똑똑하게 정함

두 알고리즘의 핵심 차이

KMP

왼쪽 → 오른쪽으로 비교
이미 맞은 문자열 안에서 반복 구조를 찾음

보이어-무어법

오른쪽 → 왼쪽으로 비교
실패한 텍스트 문자가 패턴 안에 있는지 확인
“틀린 문자를 보고 점프”

KMP는 “앞에서부터 쌓아간 일치 정보”, BM은 “뒤에서 틀린 문자를 보고 크게 이동”임.

1. KMP 문자열 검색 알고리즘

```
txt = "ZABCABXABCABD"
```

```
pat = "ABCABD"
```

핵심: 패턴 안의 반복 구조를 미리 기억해 두었다가,
실패했을 때 적절한 위치로 되돌림

KMP 코드의 두 단계

```
# 1단계: 패턴만 분석  
pt = 1; pp = 0  
while pt != len(pat):  
    ...  
    skip[pt] = pp
```

1단계의 pt, pp
둘 다 패턴 내부 위치

```
# 2단계: 실제 검색  
pt = pp = 0  
while pt != len(txt) and pp != len(pat):  
    ...  
    pp = skip[pp]
```

2단계의 pt, pp
pt: 텍스트 위치 / pp: 패턴 위치

따라서 KMP는 “skip 배열 만들기”와 “skip을 이용해 검색하기”를 분리해서 설명해야 함.

KMP 예제 문자열 확인

텍스트 txt

Z	A	B	C	A	B	X	A	B	C	A	B	D
0	1	2	3	4	5	6	7	8	9	10	11	12

패턴 pat

A	B	C	A	B	D
0	1	2	3	4	5

정답 위치
Python index = 7
사람 기준 = 8번째

바로 답을 보지 않고, KMP가 7번 위치를 찾는 과정을 pt, pp, skip 흐름으로 추적함

KMP의 skip 배열 의미

```
pat = 'ABCABD'  
skip = [0] * (len(pat) + 1)
```

skip index	0	1	2	3	4	5	6
본 패턴	-	A	AB	ABC	ABCA	ABCAB	ABCABD
skip 값	0	0	0	0	1	2	0

skip은 문자 위치가 아니라 “**몇 글자까지 봤는가**”를 저장함.
따라서 0글자부터 len(pat)글자까지 필요하므로 len(pat)+1칸 생성

왜 $\text{skip}[4] = 1$ 인가?



앞쪽 A

뒤쪽 A

ABCA에서
앞부분: A
뒷부분: A
같은 길이: 1

따라서 $\text{skip}[4] = 1$ 임. 단, $\text{skip}[4]$ 는 $\text{pat}[4]$ 의 정보가 아니라 “앞에서 4글자까지 본 결과”임.

왜 `skip[5] = 2`인가?



앞쪽 AB

뒤쪽 AB

AB CAB에서
앞부분: AB
뒷부분: AB
같은 길이: 2

검색 중 ABCAB까지 맞고 다음 글자에서 실패하면, 마지막 AB를 패턴의 처음 AB로 재활용하여 pp를 2로 되돌림

skip 배열 만드는 코드 읽기

```
def kmp_match(txt, pat):  
    pt = 1    # txt를 따라가는 커서  
    pp = 0    # pat를 따라가는 커서  
    skip = [0] * (len(pat) + 1)  
    # 건너뛰기 표  
    # 앞에서부터 pt글자까지 봤을 때의 정보  
  
    # 건너뛰기 표 만들기  
    skip[pt] = 0  
    while pt != len(pat):  
        if pat[pt] == pat[pp]:  
            pt += 1  
            pp += 1  
            skip[pt] = pp  
        elif pp == 0:  
            pt += 1  
            skip[pt] = pp  
        else:  
            pp = skip[pp]
```

문자가 같으면: 겹치는 길이가 늘어남 → pt, pp 모두 증가

문자가 다르고 pp==0이면: 겹침이 없음 → pt만 증가

문자가 다르고 pp!=0이면: 더 짧은 겹침 후보로 pp를 되돌림

결과: [0, 0, 0, 0, 1, 2, 0]

검색 단계의 변수 의미

```
# 검색하기
pt = pp = 0
while pt != len(txt) and pp != len(pat):
    if txt[pt] == pat[pp]: # 문자가 같으면 ,pt, pp 둘 다 오른쪽 이동
        pt += 1
        pp += 1
    elif pp == 0: # pp가 0이면, txt의 pt를 1 증가
        pt += 1
    else: # 문자가 다른데 pp != 0이면 skip 사용 즉, 패턴의 앞부분은 어느 정도
        맞았고, 중간에서 틀린 상황
        pp = skip[pp]

return pt - pp if pp == len(pat) else -1
```

pt = 텍스트 txt의 현재 위치

pp = 패턴 pat의 현재 위치

불일치가 발생하면 pp가 0인지 아닌지가 핵심임

KMP 검색 규칙 ① 문자가 같으면 둘 다 이동

Z	A	B	C	A	B	X
0	1	2	3	4	5	6

A	B	C	A	B	D
0	1	2	3	4	5

```
if txt[pt] == pat[pp]:  
    pt += 1  
    pp += 1
```

예: txt[1]=A, pat[0]=A
일치 → pt: 1→2, pp: 0→1

KMP 검색 규칙 ② 첫 글자부터 틀리면 txt만 이동

Z	A	B	C	A	B	X
0	1	2	3	4	5	6

A	B	C	A	B	D
0	1	2	3	4	5

```
elif pp == 0:  
    pt += 1
```

예: txt[0]=Z, pat[0]=A
불일치 + pp==0
→ pt만 1 증가

중요 실패 지점: $pt = 6$, $pp = 5$

Z	A	B	C	A	B	X	A	B	C	A	B	D
0	1	2	3	4	5	6	7	8	9	10	11	12

A	B	C	A	B	D
0	1	2	3	4	5

$txt[6] = X$

$pat[5] = D$

$X \neq D$

$pp = 5$ 라는 것은 $pat[0] \sim pat[4]$, 즉 ABCAB까지 이미 맞았다는 뜻임.
따라서 완전히 처음으로 돌아가지 않고 $skip[5]$ 를 확인함.

pp = skip[pp]가 실행되는 순간

```
else:  
    pp = skip[pp]  
  
# 현재 pp = 5  
pp = skip[5] # 2
```

실행 전
pt=6, pp=5

실행 후
pt=6, pp=2



ABCAB 안에서 앞쪽 AB와 뒤쪽 AB가 같으므로
 $\text{skip}[5] = 2$

왜 pp가 2가 되는가?



앞쪽 AB

뒤쪽 AB



pp = 2

KMP의 판단

- “마지막 AB는 패턴의 처음 AB와 같다. 그러므로 AB는 다시 비교하지 말고, 그 다음 C부터 비교하자.”

pp가 2가 된 뒤에도 X는 아직 안 맞음

Z	A	B	C	A	B	X	A	B	C	A	B	D
0	1	2	3	4	5	6	7	8	9	10	11	12

A	B	C	A	B	D
0	1	2	3	4	5

txt[6] = X, pat[2] = C → 불일치

```
pp = skip[2]  
pp = 0
```

여전히 pt는 6임. KMP는 가능한 재활용을 모두 확인한 뒤 텍스트 위치를 넘김

pp가 0이 되면 txt를 한 칸 넘김

Z	A	B	C	A	B	X	A	B	C	A	B	D
0	1	2	3	4	5	6	7	8	9	10	11	12

A	B	C	A	B	D
0	1	2	3	4	5

`txt[6] = X, pat[0] = A`
불일치 + `pp == 0`

```
elif pp == 0:  
    pt += 1
```

결과: `pt 6 → 7, pp 0 유지`

두 번째 위치에서 완전히 일치

Z	A	B	C	A	B	X	A	B	C	A	B	D
0	1	2	3	4	5	6	7	8	9	10	11	12

A	B	C	A	B	D
0	1	2	3	4	5

txt[7] A = pat[0] A
txt[8] B = pat[1] B
txt[9] C = pat[2] C

txt[10] A = pat[3] A
txt[11] B = pat[4] B
txt[12] D = pat[5] D

최종 상태: pt = 13, pp = 6

KMP 반환값 pt - pp의 의미

```
return pt - pp if pp == len(pat) else -1
```

Z	A	B	C	A	B	X	A	B	C	A	B	D
0	1	2	3	4	5	6	7	8	9	10	11	12

검색 성공 조건: $pp == \text{len}(\text{pat})$

$pt - pp = 13 - 6 = 7$
반환값 7 = Python index 기준 시작 위치

출력에서 $\text{idx} + 1$ 을 하면 사람이 세는 기준인 8번째가 됨.

KMP 전체 흐름 요약

① skip 만들기

패턴 내부에서 앞부분과 뒷부분이 같은 길이를 기록

② 검색 시작 : 같으면 pt, pp를 함께 증가

③ 실패 처리 : pp가 0이 아니면 pp = skip[pp]

```
pt=6, pp=5: X != D  
pp = skip[5] = 2
```

```
pt=6, pp=2: X != C  
pp = skip[2] = 0
```

```
pt=6, pp=0: X != A  
pt += 1 → pt=7
```

KMP는 “이미 맞은 부분을 버리지 않고, 패턴 안의 반복 구조를 이용해 pp를 되돌리는 알고리즘”임.

2. 보이어-무어법

```
txt = "ABABCDEFGGHA"
```

```
pat = "ABC"
```

핵심: 패턴의 오른쪽 끝부터 비교하고,
실패한 문자를 기준으로 pt를 크게 이동함.

보이어-무어법

일반적인 왼쪽 → 오른쪽 비교



A → B → C 순서

보이어-무어: 오른쪽 → 왼쪽 비교



C → B → A 순서

오른쪽 끝 문자가 틀리면, 그 문자가 패턴 안에 있는지 보고 패턴을 크게 이동할 수 있음

BM 예제

보이어 무어법

```
def bm_match(txt, pat):
```

```
    skip = [None] * 256 # 건너뛰기 표 생성, 알파벳 포함 아스키 문자 256개 담을 수 있는 크기
```

```
    for pt in range(256):
```

```
        skip[pt] = len(pat) # skip 배열에 패턴의 길이만큼 건너뛰기 값 설정
```

```
        # 패턴 안에 없는 문자를 만나면 패턴 길이만큼 통째로 건너뛴다.
```

```
    for pt in range(len(pat)):
```

```
        skip[ord(pat[pt])] = len(pat) - pt - 1
```

```
        # ord()는 문자를 숫자로 바꿔서 리스트 인덱스로 사용하게
```

검색하기

```
while pt < len(txt): # 본문 끝까지 비교
```

```
    pp = len(pat) - 1 # 뒤에서부터 거꾸로 비교, 패턴의 커서를 맨 뒤로 보내버림
```

```
    while txt[pt] == pat[pp]: # 본문과 패턴의 문자가 같다면 실행
```

```
        if pp == 0: # pp가 0까지 온건, 뒤에서부터 시작해 패턴의 첫 글자까지
```

```
            # 완벽하게 똑같다는 뜻, 문자열 찾기에 성공. pt를 반환하고 종료
```

```
            return pt
```

```
        pt -= 1 # 본문과 패턴이 같으면 계속 감소
```

```
        pp -= 1 # 본문과 패턴이 같으면 계속 감소
```

```
    pt += skip[ord(txt[pt])] if skip[ord(txt[pt])] > len(pat) - pp else len(pat) - pp
```

```
    # 불일치가 났을 때 , pt를 몇 칸 오른쪽으로 이동할 것인가?
```

```
    # move1 = skip[ord(txt[pt])] : txt에서 불일치가 난 문자 찾아서 그것에 해당하는 길이만큼 이동하라
```

```
    # move2 = len(pat) - pp # 최소 이동량 보정값, 최소 1칸은 이동해라. 왜냐하면 skip이 0이 나오는 경우가 있어서
```

```
    # if move1 > move2:
```

```
        #     pt += move1
```

```
    # else:
```

```
        #     pt += move2
```

```
return -1
```

BM 예제 문자열과 목표

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

txt = "ABABCFGHGA"

A	B	C
0	1	2

정답 시작 위치
Python index = 2
출력: idx + 1 → 3번째

① skip 표 만들기 → ② 패턴 끝 문자부터 비교 → ③ 불일치 시 이동량 계산 → ④ 일치하면 왼쪽으로 계속 비교

BM의 skip 표: 문자별 이동 거리

```
skip = [None] * 256

for pt in range(256):
    skip[pt] = len(pat)

for pt in range(len(pat)):
    skip[ord(pat[pt])] = len(pat) - pt - 1
```

문자	A	B	C	그 외
끝에서 거리	2	1	0	3

그 외 문자 = 패턴에 없는 문자이므로
패턴 길이만큼 이동함.

skip 표 계산을 직접 해보기

A	B	C
0	1	2

```
skip[ord(pat[pt])] = len(pat) - pt - 1
```

$pt = 0 \rightarrow pat[0] = A \rightarrow 3 - 0 - 1 = 2$

$pt = 1 \rightarrow pat[1] = B \rightarrow 3 - 1 - 1 = 1$

$pt = 2 \rightarrow pat[2] = C \rightarrow 3 - 2 - 1 = 0$

결과: A는 2칸, B는 1칸, C는 0칸, 그 외는 3칸

ord()는 왜 쓰는가?

```
ord('A')    # 65  
ord('B')    # 66  
ord('C')    # 67  
chr(65)     # 'A'
```

```
skip[ord('A')] = 2  
# 실제 의미: skip[65] = 2  
  
skip[ord('B')] = 1  
# 실제 의미: skip[66] = 1
```

리스트는 skip[65]처럼 숫자 인덱스로 접근함.

skip["A"]처럼 문자 인덱스를 직접 사용할 수 없으므로 ord()로 문자를 숫자 코드로 바꿈

패턴에 같은 문자가 두 번 나오면?

A	B	C	A
0	1	2	3

```
pat = 'ABCA'
A = 4 - 0 - 1 = 3
B = 4 - 1 - 1 = 2
C = 4 - 2 - 1 = 1
A = 4 - 3 - 1 = 0
```

계산 과정에서는 A=3이 먼저 나오지만, 마지막 A=0이 앞의 A 값을 덮어씀

최종 skip: A=0, B=2, C=1, 그 외=4

단, 실제 이동은 $\max(\text{skip값}, \text{len(pat)} - \text{pp})$ 를 사용하므로 0칸 이동으로 멈추지 않음

BM 검색 단계의 변수 pt와 pp

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

A	B	C
0	1	2

$pt = 2$
텍스트에서 현재 비교 위치

$pp = 2$
패턴에서 현재 비교 위치

$pt = \text{len}(\text{pat}) - 1$

BM 검색 부분 코드: 큰 흐름

```
while pt < len(txt):
    pp = len(pat) - 1
    while txt[pt] == pat[pp]:
        if pp == 0:
            return pt
        pt -= 1
        pp -= 1
    pt += skip[ord(txt[pt])] if skip[ord(txt[pt])] > len(pat) - pp else
len(pat) - pp
return -1
```

- ① pt가 텍스트 끝을 넘기 전까지 반복
- ② pp는 매번 패턴의 마지막 위치에서 시작
- ③ 문자가 같으면 pt와 pp를 왼쪽으로 이동
- ④ pp가 0이면 패턴 전체 일치
- ⑤ 다르면 이동량을 계산해 pt를 오른쪽으로 이동

첫 번째 비교: A와 C가 다르다

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

A	B	C
0	1	2

txt[2] = A
pat[2] = C
A ≠ C

현재 상태
pt = 2, pp = 2, len(pat)=3

이동량 계산 한 줄 풀어쓰기

```
pt += skip[ord(txt[pt])] if skip[ord(txt[pt])] > len(pat) - pp else len(pat) - pp
```

```
move1 = skip[ord(txt[pt])]
move2 = len(pat) - pp

if move1 > move2:
    pt += move1
else:
    pt += move2
```

```
# 같은 의미
pt += max(skip[ord(txt[pt])], len(pat) - pp)
```

즉, 문자 기준 이동량과 위치 기준 최소 이동량
중 더 큰 값을 사용함.

move1: 문자 기준 이동량

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

```
move1 = skip[ord(txt[pt])]
       = skip[ord('A')]
       = skip[65]
       = 2
```

A	B	C
0	1	2

의미: 텍스트의 A와 패턴 안의 A를 맞추려면 2칸 이동함.

move2: 위치 기준 최소 이동량

```
pp = 2  
len(pat) = 3  
move2 = len(pat) - pp  
        = 3 - 2  
        = 1
```

의미: 최소 1칸은 오른쪽으로 이동해야 같은 자리 반복을 피함.

패턴 끝 문자의 skip 값은 0일 수 있음.

예: pat="ABC"에서 skip["C"] = 0
불일치했는데 0칸 이동하면 같은 위치를 계속 비교할 수 있으므로 move2와 비교함.

첫 이동은 2칸이다

$\text{move1} = \text{skip}["A"] = 2$

$\text{move2} = 3 - 2 = 1$

$\text{max}(2, 1) = 2$

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

이동 전

A	B	C
0	1	2

2칸 이동 후

A	B	C
0	1	2

$\text{pt} = 2 \rightarrow 4$

두 번째 비교: C가 맞으면 왼쪽으로 간다

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

A	B	C
0	1	2

txt[4] = C, pat[2] = C
일치

```
if pp == 0:  
    return pt  
pt -= 1  
pp -= 1
```

아직 pp=2이므로 시작까지 간 것은 아님. 따라서 왼쪽으로 한 칸 이동함.

B도 맞고, 다시 왼쪽으로 이동

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

A	B	C
0	1	2

txt[3] = B, pat[1] = B
일치

상태 변화
C 일치 후: pt=3, pp=1
B도 일치 → pt=2, pp=0

A까지 맞으면 검색 성공

A	B	A	B	C	D	E	F	G	H	A
0	1	2	3	4	5	6	7	8	9	10

A	B	C
0	1	2

txt[2] = A, pat[0] = A
일치

```
if pp == 0:  
    return pt
```

오른쪽에서 왼쪽으로 C → B → A가 모두 맞음 이때 현재 pt는 패턴의 첫 문자 위치이므로 return pt = 2임.

BM 전체 추적표

단계	pt	pp	비교	결과	다음 동작
1	2	2	txt[2]=A vs pat[2]=C	불일치	max(2,1)=2칸 이동
2	4	2	txt[4]=C vs pat[2]=C	일치	왼쪽 이동
3	3	1	txt[3]=B vs pat[1]=B	일치	왼쪽 이동
4	2	0	txt[2]=A vs pat[0]=A	일치	return 2

출력 코드에서 `print(idx + 1)`이므로 최종 출력은 3임.